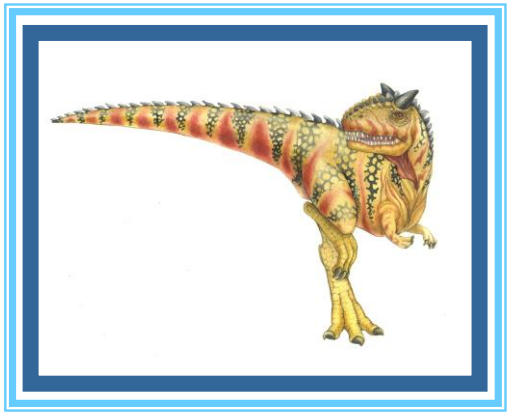
Chapter 10: Virtual Memory



Silberschatz, Galvin and Gagne ©2018



Antecedentes

- El código debe estar en la memoria para ejecutarse, pero rara vez se utiliza el programa completo.
  - Código de error, rutinas inusuales, grandes estructuras de datos.
  - No se necesita el código completo del programa al mismo tiempo
  - Considere la capacidad de ejecutar un programa parcialmente cargado.
  - El programa ya no está limitado por los límites de la memoria física.
  - Cada programa ocupa menos memoria mientras se ejecuta -> se ejecutan más programas al mismo tiempo.
    - Mayor utilización y rendimiento de la CPU sin aumentar el tiempo de respuesta ni el tiempo de permanencia.
    - Se necesita menos E/S para cargar o intercambiar programas en la memoria -> cada programa de usuario se ejecuta más rápido



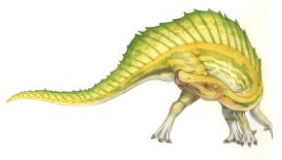


Memoria Virtual

- **Memoria Virtual**

  Separación de la memoria lógica del usuario de la memoria física.

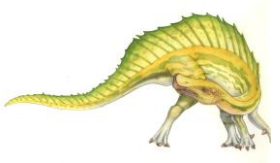
  - Sólo una parte del programa necesita estar en la memoria para su ejecución.
  - Por lo tanto, el espacio de direcciones lógicas puede ser mucho mayor que el espacio de direcciones físicas.
  - Permite que varios procesos compartan espacios de direcciones
  - Permite una creación de procesos más eficiente
  - Más programas ejecutándose simultáneamente
  - Menos E/S necesarias para cargar o intercambiar procesos





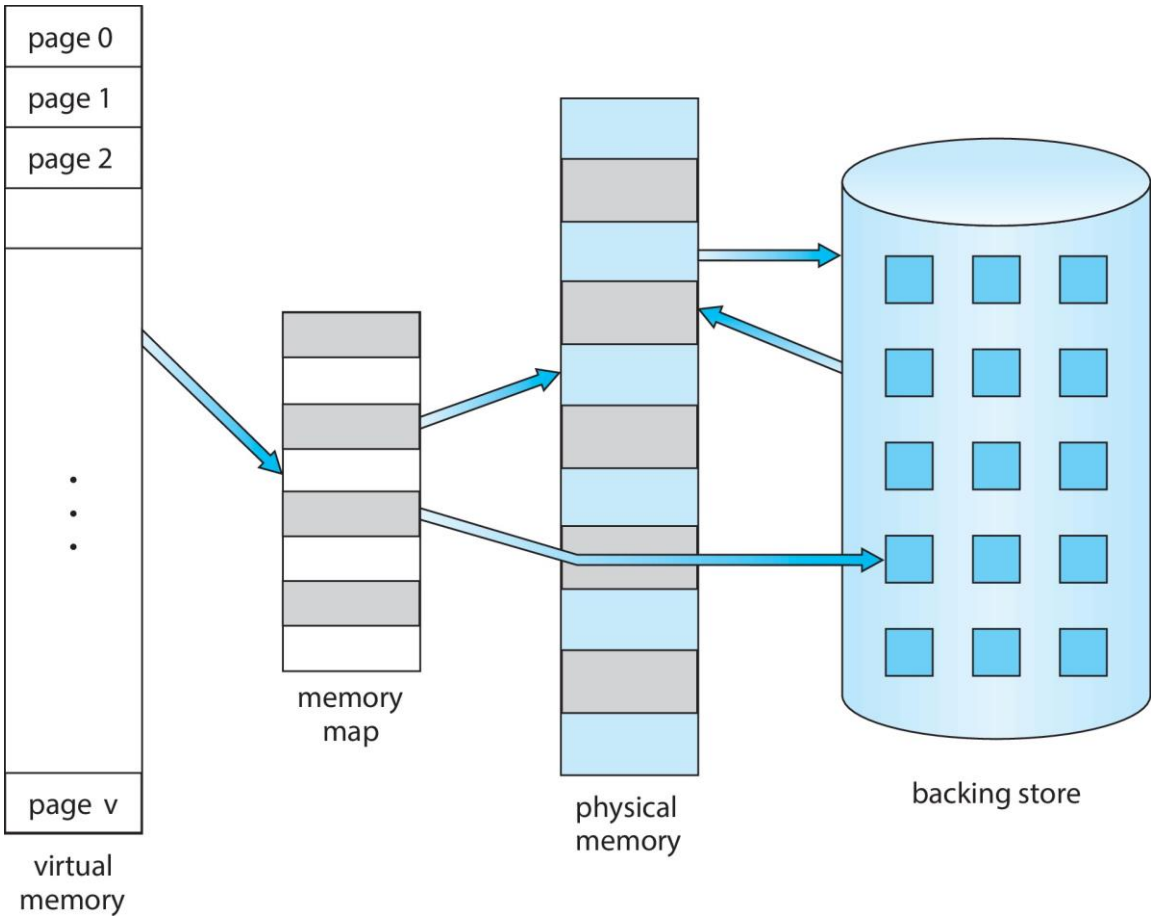
Memoria Virtual (Cont.)

- Espacio de direcciones virtuales: vista lógica de cómo se almacena el proceso en la memoria
  - Generalmente comienza en la dirección 0, direcciones contiguas hasta el final del espacio.
  - Mientras tanto, la memoria física está organizada en marcos de páginas.
  - El MMU debe asignar lo lógico a lo físico.
- La memoria virtual se puede implementar mediante:
  - Paginación de demanda
  - Segmentación de la demanda





Memoria virtual: Más grande que la memoria física

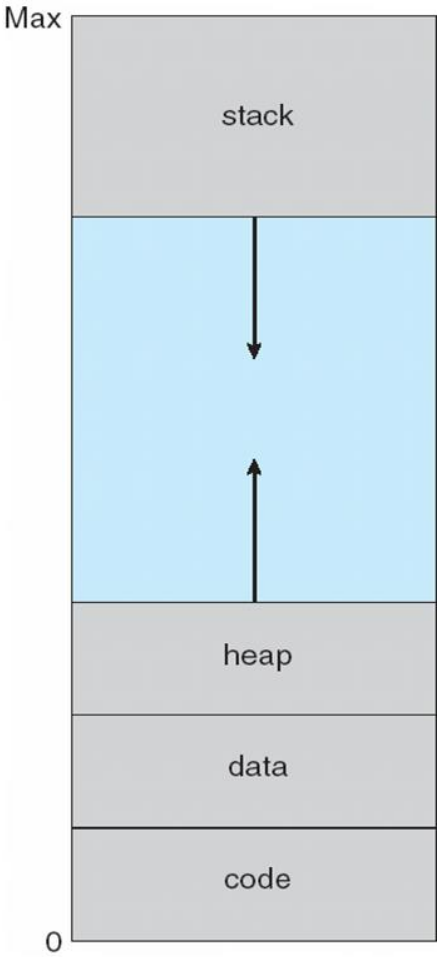






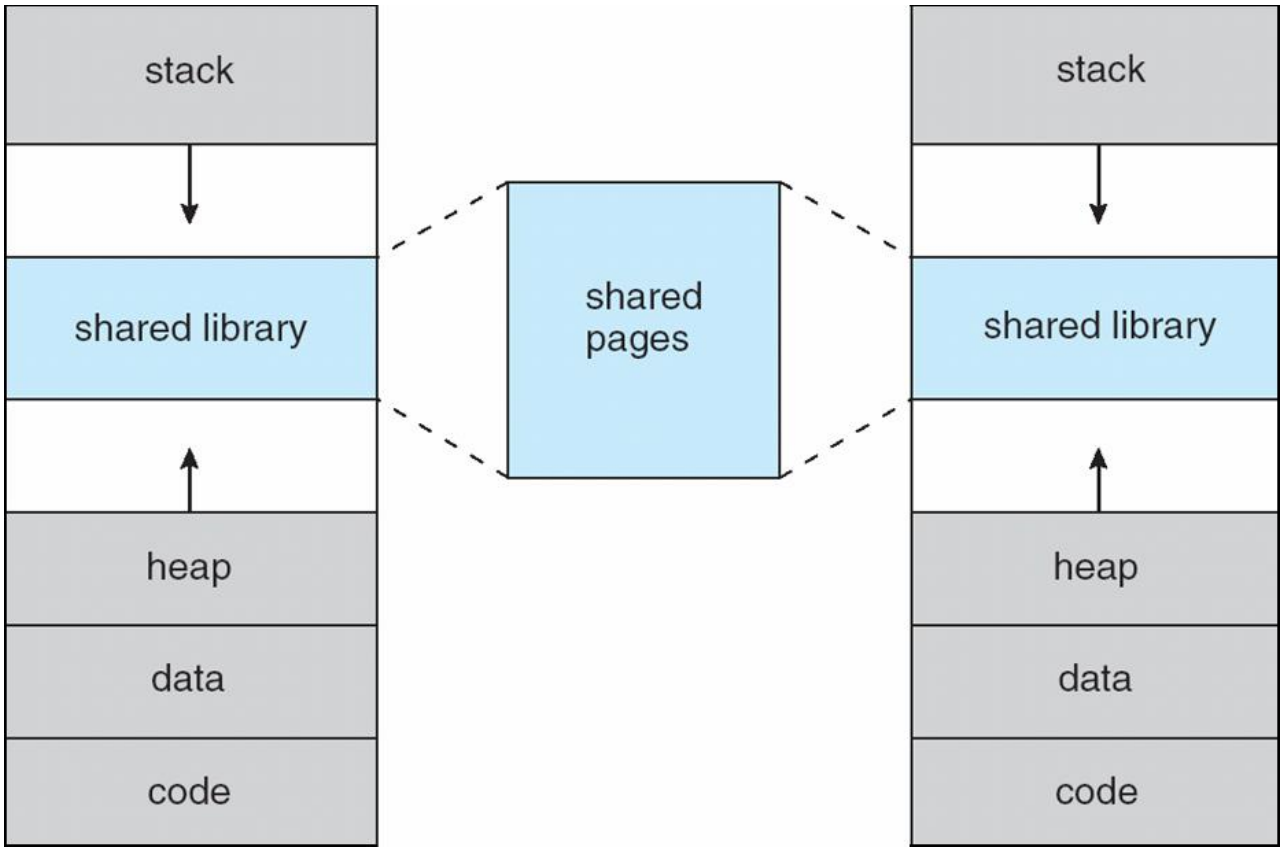
Espacio de direcciones virtuales

- Por lo general, diseña un espacio de direcciones lógicas para que la pila comience en la dirección lógica máxima y crezca "hacia abajo" mientras que el resto crece "hacia arriba".
  - Maximiza el uso del espacio de direcciones
  - El espacio de direcciones no utilizado entre los dos se determina como vacío.
  - No se necesita memoria física hasta que el resto o la pila crezcan hasta una página nueva determinada
  - Permite espacios de direcciones dispersas con vacíos para el crecimiento, bibliotecas vinculadas dinámicamente, entre otras.
  - El Sistema de Librerías es compartida mediante mapeo en el espacio de direcciones virtuales
  - La Memoria es compartida mediante la asignación de páginas de lectura y escritura en el espacio de direcciones virtuales
  - Las páginas se pueden compartir durante la ejecución de la instrucción fork(), lo que acelera la creación del proceso





Biblioteca compartida: usando Memoria Virtual





Paginación por demanda

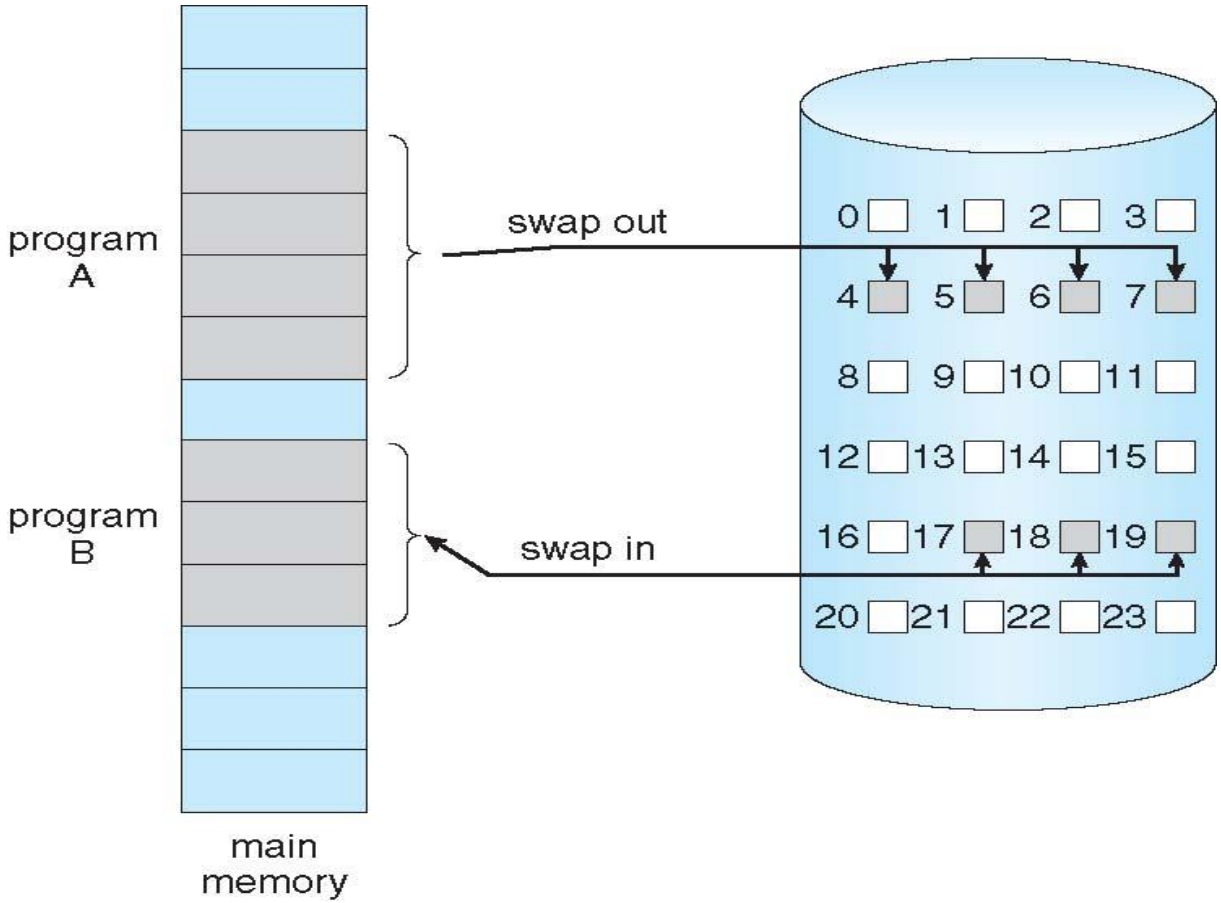
- Podría traer todo el proceso a la memoria en el momento de la carga.
- O traer una página a la memoria sólo cuando sea necesario
  - Se necesitan menos E/S, sin E/S innecesarias
  - Se necesita menos memoria
  - Respuesta más rápida
  - Más usuarios
- Similar al sistema de paginación con intercambio (diagrama en el próximo slides)
- Se necesita página => referencia a ella
- referencia no válida => cancelar
- no en memoria => traer a la memoria

- **Lazy swapper (**intercambiador diferido): nunca intercambia una página en la memoria a menos que sea necesaria
  - Swapper que se ocupa de páginas, es un busca página.





Paginación por Demanda







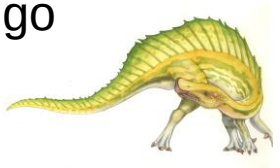
Conceptos Básicos

- Con el intercambio, el busca página supone qué páginas se utilizarán antes de volver a intercambiarlas.
- ¿Cómo determinar ese conjunto de páginas?

  Necesita una nueva funcionalidad el MMU para implementar la paginación de demanda

- Si las páginas necesarias ya están residentes en la memoria
  - No hay diferencia con la paginación sin demanda

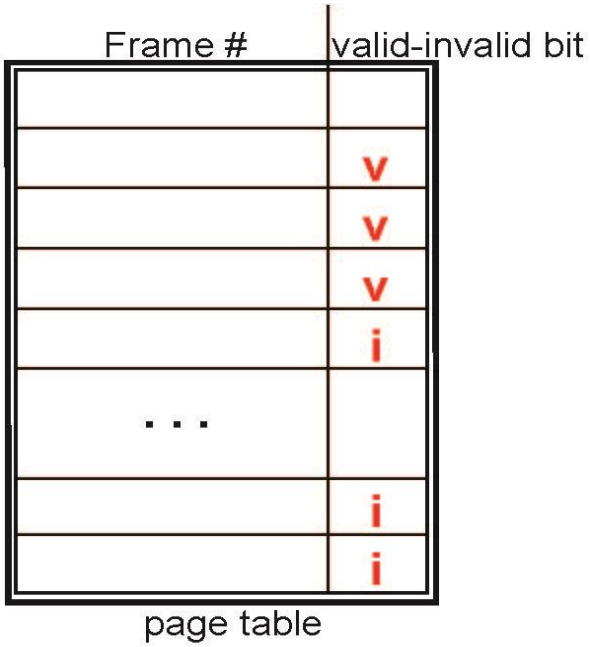
- Si la página es necesaria y no reside en la memoria
  - Necesidad de detectar y cargar la página en la memoria desde el almacenamiento.
    - Sin cambiar el comportamiento del programa
    - Sin necesidad de que el programador cambie el código





Bit válido no válido

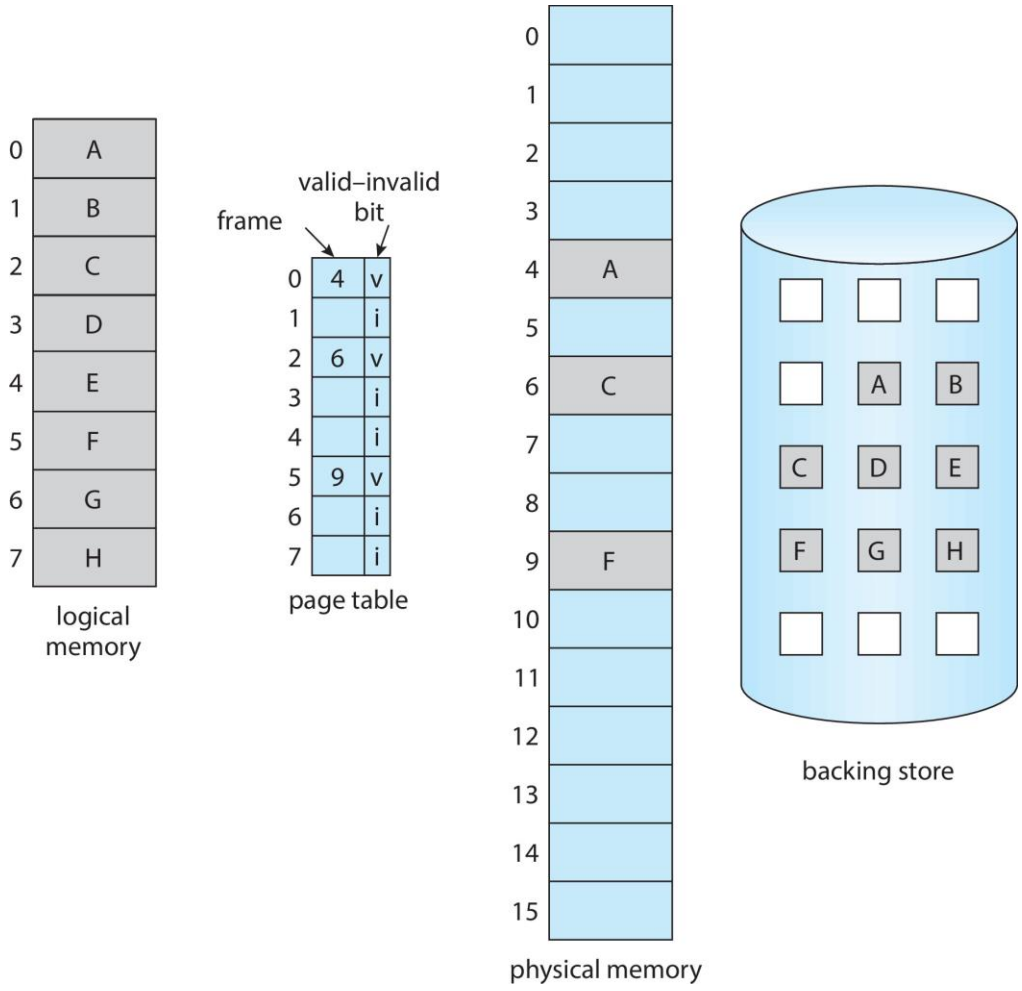
- Con cada entrada de la tabla de páginas se asocia un bit válido-inválido: (v => en memoria – residente en memoria, i => no en memoria)
- El bit inicialmente válido-inválido se establece en i en todas las entradas
- Ejemplo:



- Durante la traducción de direcciones MMU, si el bit válido-inválido en la entrada de la tabla de páginas es i ⇒ Fallo de página



Tabla de páginas cuando algunas páginas no están en la memoria principal





Pasos para manejar el Fallo de página

1. Si hay una referencia a una página, la primera referencia a esa página será un trap en el sistema operativo.

   Fallo de página

2. El sistema operativo mira la tabla para decidir:

   Referencia no válida $\Rightarrow$ cancelar, o

   Simplemente no está en la memoria

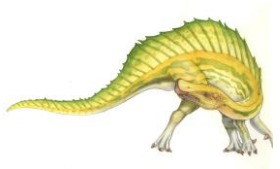
3. Encontrar marco libre.

4. Intercambiar página en marco mediante operación de disco Planificada (en base al algoritmo de búsqueda que usa el dispositivo).

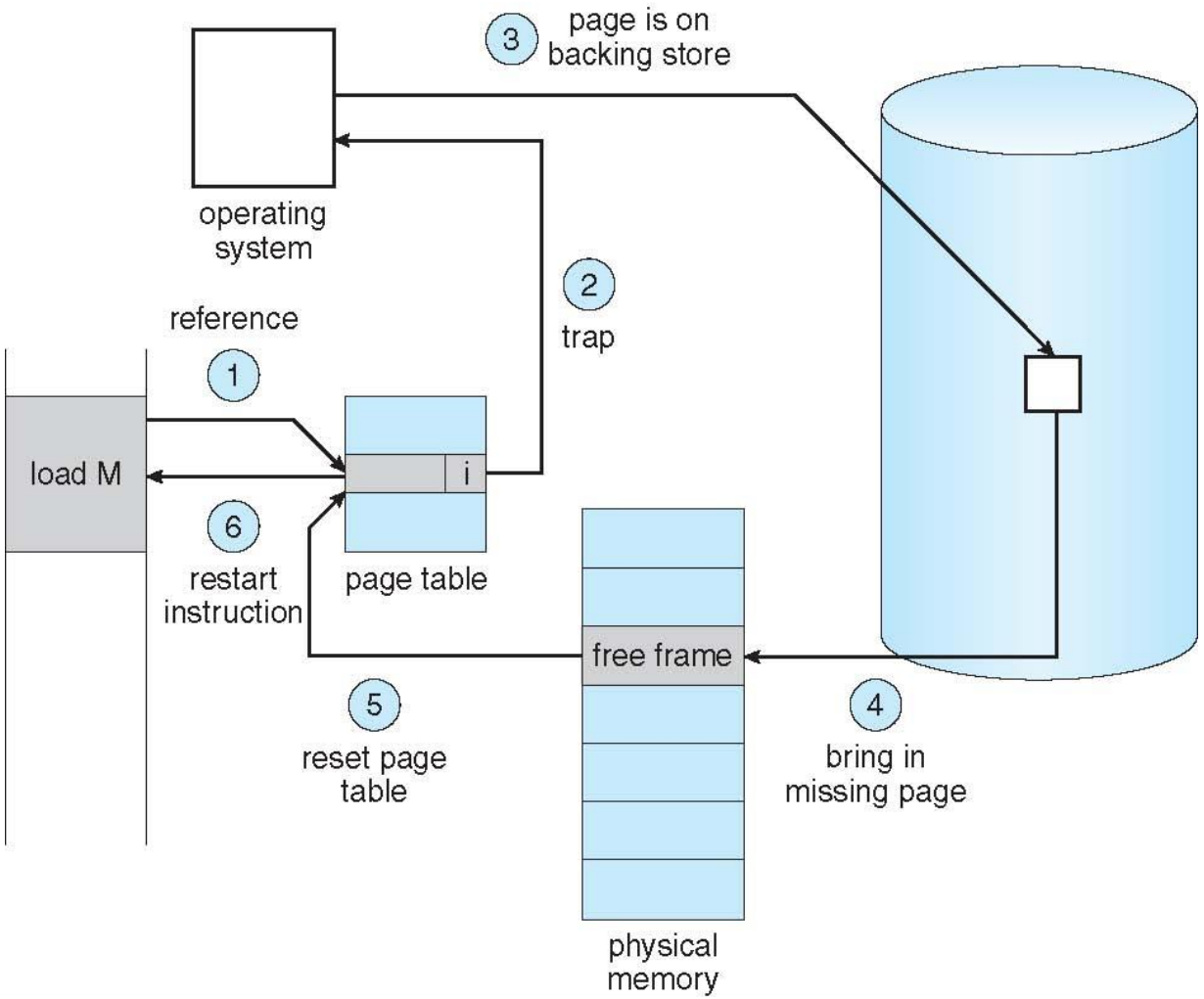
5. Restablecer tablas para indicar que la página está ahora en la memoria

   Establecer bit de validación = v

   Reinicia la instrucción que causó el Fallo de página.



Pasos para manejar el Fallo de página(Cont.)



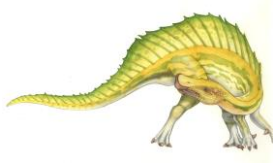


Aspectos de la Demanda de Páginas

- Caso extremo: iniciar el proceso sin páginas en la memoria
  - El sistema operativo establece un puntero en la primera instrucción del proceso, no residente en la memoria => Fallo de página
  - Y para todas las demás páginas de proceso en el primer acceso
  - Paginación de demanda pura

- Una instrucción determinada podría acceder a varias páginas -> Fallas en varias páginas
  - Considera aplicar el ciclo fetch y decodificar una instrucción que suma 2 números de la memoria y almacena el resultado en la memoria.

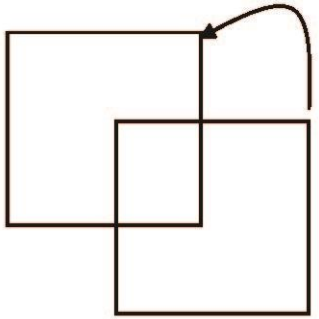
- Soporte de hardware necesario para paginación bajo demanda
  - Tabla de páginas con bit válido/inválido
  - Memoria secundaria (dispositivo de intercambio con espacio de intercambio)
  - Reinicio de instrucción



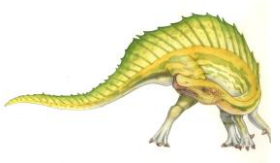


Reinicio de instrucciones

- Considere una instrucción que podría acceder a varias ubicaciones diferentes.
  - Movimiento de bloque



  - Ubicación de incremento/disminución automática
  - ¿Reiniciar toda la operación?
  - ¿Qué pasa si el origen y el destino se superponen?





Lista de Marcos Libres (Mapa de Bits)

- Cuando ocurre una Falla de página, el sistema operativo debe llevar la página deseada desde el almacenamiento secundario a la memoria principal.
- La mayoría de los sistemas operativos mantienen una lista de marcos libres: un conjunto de marcos libres para satisfacer dichas solicitudes. (o en su defecto un vector, cuya posición está asociada a cada marco de página, indicando si esta usado=> 1 o no usado=>0).



- El sistema operativo normalmente asigna a los marcos libres utilizando una técnica conocida como relleno cero bajo demanda: el contenido de los marcos de página se pone en cero antes de ser asignados.
- Cuando se inicia un sistema, toda la memoria disponible se coloca en la lista de marcos libres.





Etapas en la paginación bajo demanda: peor caso

1. Se produce un trap al sistema operativo
2. Guarda los registros de usuario y el estado del proceso.
3. Determina que la interrupción fue una falla de página
4. Verifica que la referencia de la página sea legal y determina la ubicación de la página en el disco
5. Emite una lectura desde el disco a un marco libre:

    Espera en una cola para este dispositivo hasta que se atienda la solicitud de lectura.

    Espera la búsqueda del dispositivo y/o el tiempo de latencia

    => tiempo de acceso

    Comienza la transferencia de la página a un marco libre.

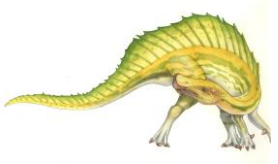
    => tiempo de transferencia





Etapas en la paginación bajo demanda: (Cont.)

6. Mientras espera, asigna la CPU a otro usuario
7. Recibe una interrupción del subsistema de E/S del disco (E/S completada)
8. Guarda los registros y el estado del proceso para el otro usuario.
9. Determina que la interrupción fue del disco.
10. Corrige la tabla de páginas y otras tablas para mostrar que la página ahora está en la memoria.
11. Espera a que la CPU se asigne nuevamente a este proceso
12. Restaura los registros de usuario, el estado del proceso y la nueva tabla de páginas, y luego
13. Reanuda la instrucción interrumpida





¿Qué pasa si no hay un marco libre?

- Utilizado por las páginas de proceso.
- También en demanda del kernel, buffers de E/S, etc.
- ¿Cuándo asignar a cada una?
- Reemplazo de página: buscar alguna página en la memoria, pero que no esté realmente en uso, =>sacar la página
  - Algoritmo: ¿terminar? ¿Intercambiar? ¿reemplazar la página?
  - Rendimiento: Un algoritmo que dé como resultado un número mínimo de errores de página
- La misma página puede ser utilizada varias veces.



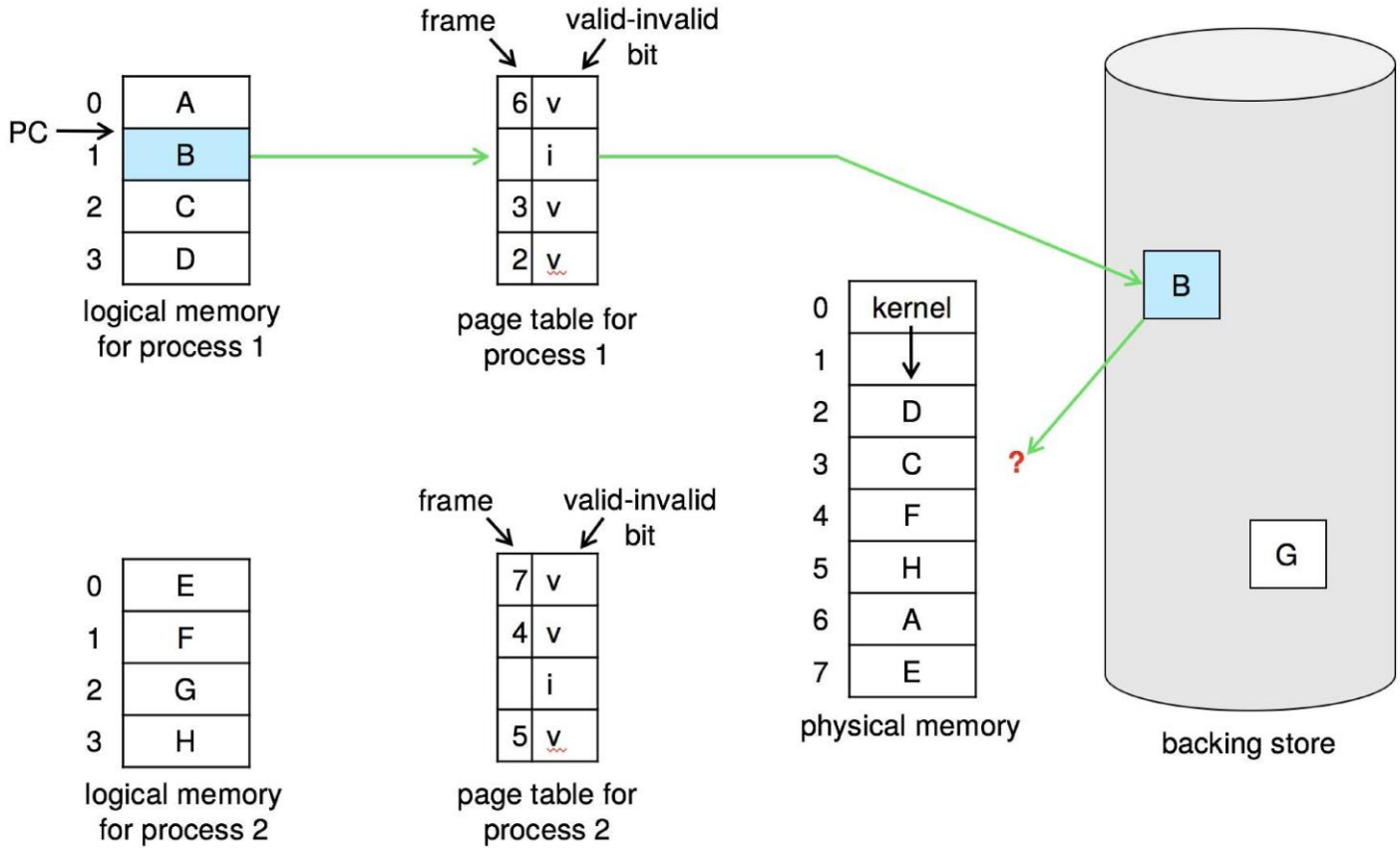


Reemplazo de Páginas

- Principios Básicos:
  - Prevenir la asignación excesiva de memoria modificando la rutina del servicio de Falla de Página para incluir el reemplazo correspondiente.
  - Utilizar un bit de modificación, para reducir la sobrecarga de las transferencias de páginas: solo las páginas modificadas se grabaran en el disco.
  - Reemplazar páginas: se puede proporcionar una memoria virtual mayor a la memoria física.



Necesidad de reemplazo de página





Reemplazo de página básico

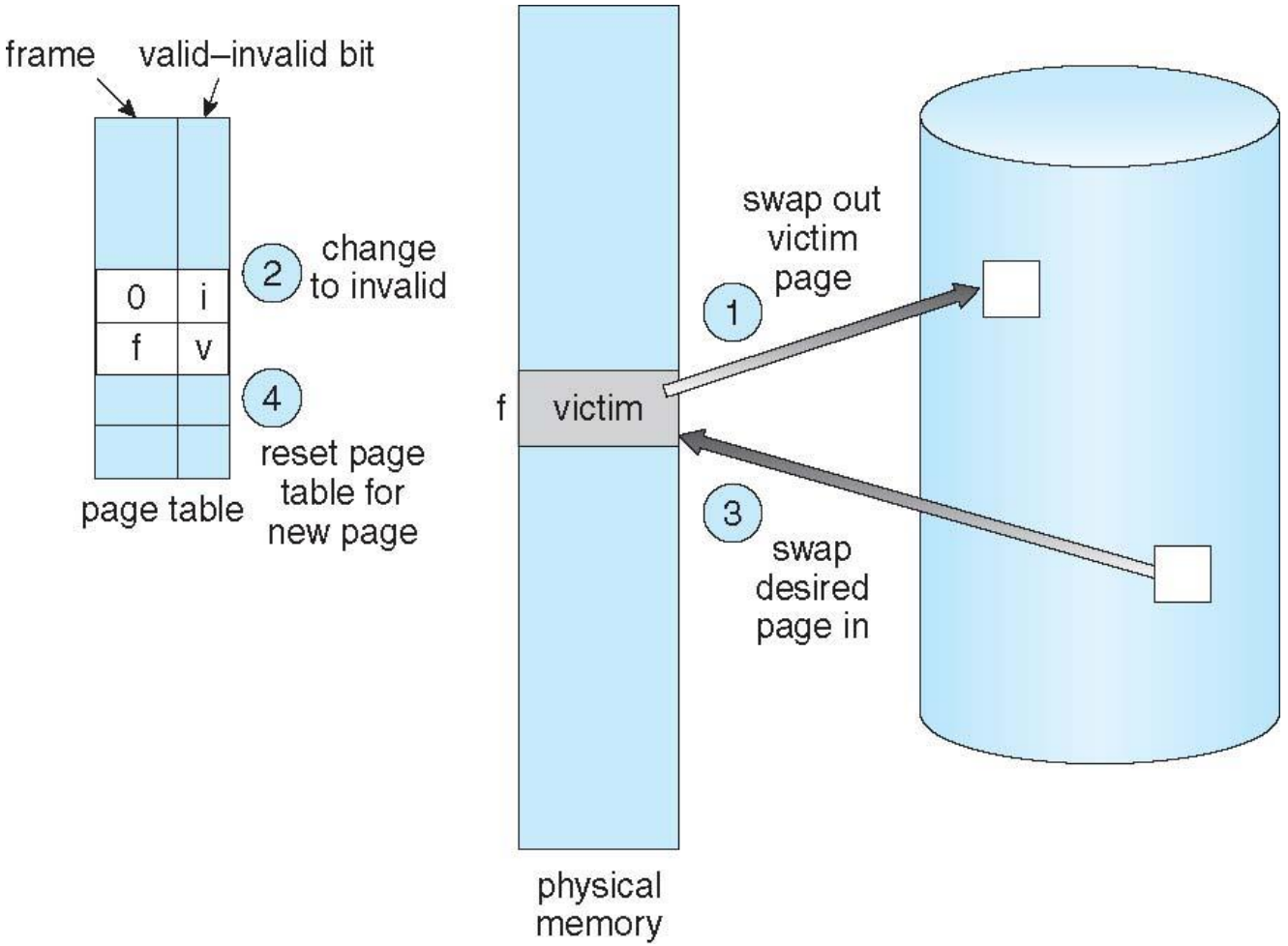
1. Encuentra la ubicación de la página deseada en el disco
2. Encuentra un marco libre: - Si hay un marco libre, úselo - Si no hay un marco libre, use un algoritmo de reemplazo de página para seleccionar un marco víctima –

   Grabar la página ubicada en el marco víctima en el disco si está Modificada.

3. Llevar la página deseada al (recién) marco libre; actualizar la tabla de páginas.
4. Continuar el proceso reiniciando la instrucción que causó el trap.



Reemplazo de página





Algoritmos de reemplazo de páginas

- **El algoritmo de localización del marco:**
  - ¿Cuántos marcos se le dará a cada proceso?
  - ¿Qué marcos reemplazar?
- **Algoritmo de reemplazo de página**
- Desear la tasa más baja de Falla de página tanto en el primer acceso como en el nuevo acceso
- Evaluar el algoritmo ejecutándolo en una cadena particular de referencias de memoria (cadena de referencia) y calculando el número de Falla de página en esa cadena.
  - La cadena son solo números de página, no direcciones completas.
  - El acceso repetido a la misma página estando en la memoria no provoca una Falla de página
  - Los resultados dependen del número de marcos disponibles.
- En todos nuestros ejemplos, la cadena de referencia de los números de página referenciados es:

7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

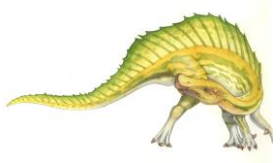
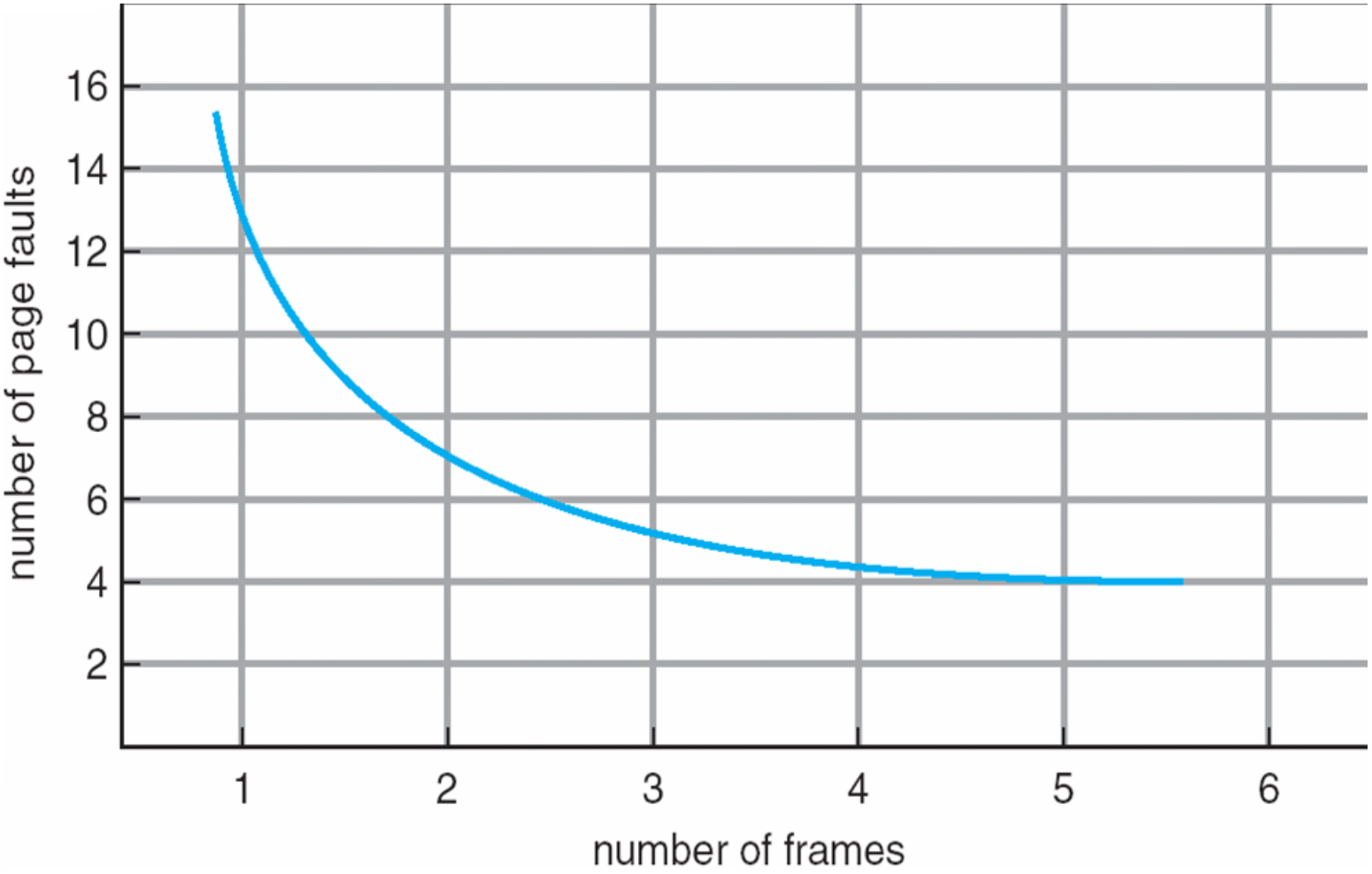




Gráfico de Fallas de página frente al número de Marcos

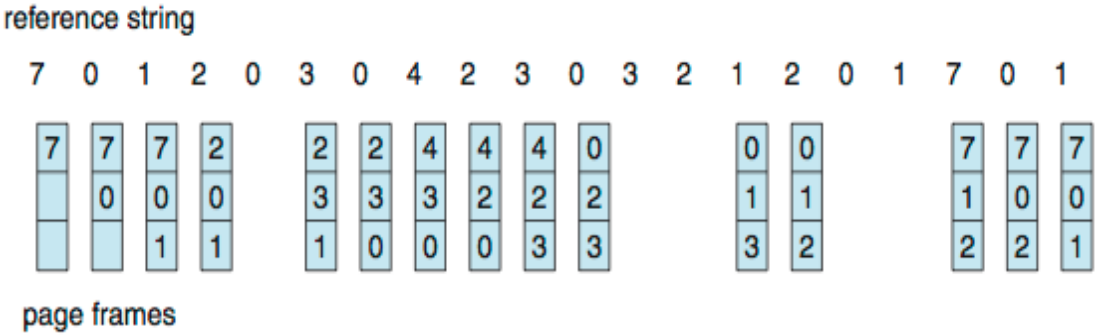






Algoritmo primero en entrar, primero en salir (FIFO)

- Cadena de referencia: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 marcos (Pueden haber 3 páginas en la memoria a la vez por proceso)



15 Fallas de Páginas

- Puede variar por cadena de referencia: considerer :1,2,3,4,1,2,5,1,2,3,4,5

  - Agregar más marcos puede causar más Fallas de página?

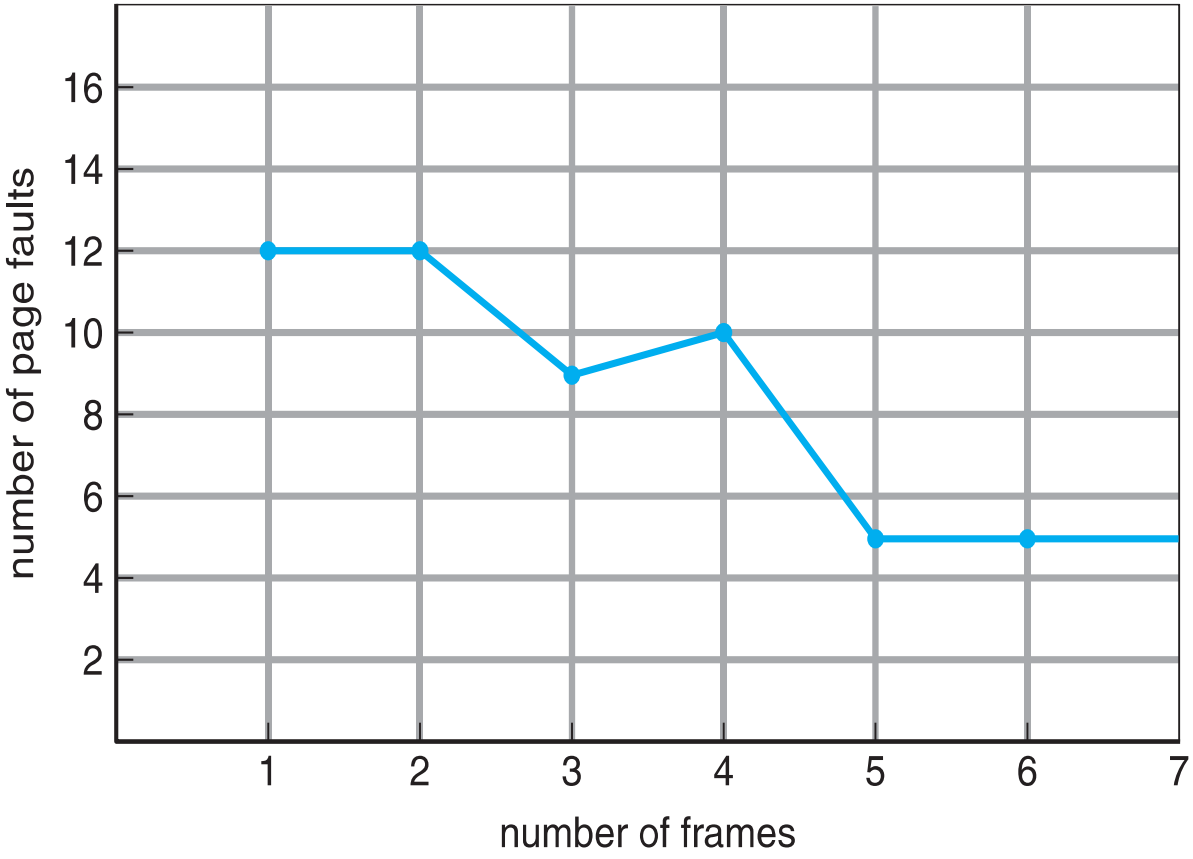
    Analizar Anomalía de Belady' s

- ¿Cómo realizar un seguimiento de la antigüedad de las páginas?

  Simplemente use una cola FIFO



FIFO que muestra la anomalía de Belady



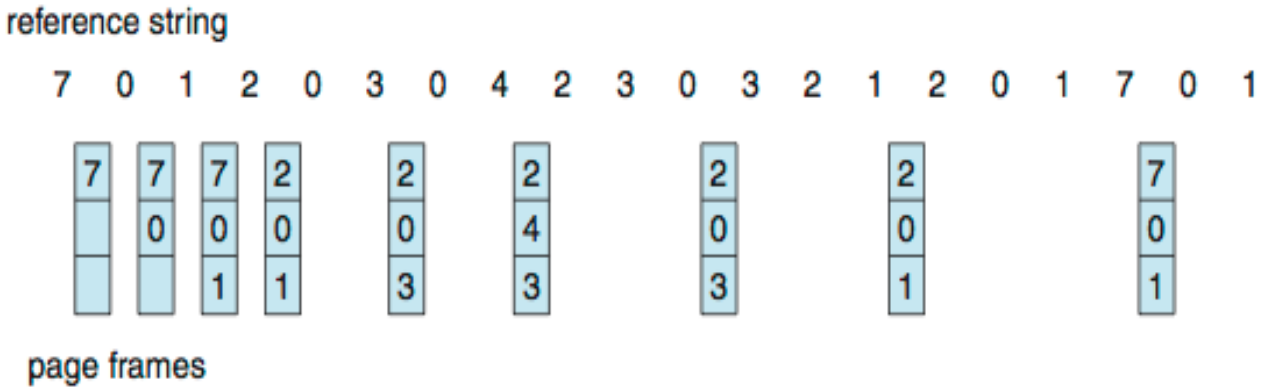


Algoritmo óptimo

- Reemplazar la página que no se utilizará durante un período de tiempo más largo

  9 Fallas de página es óptimo para el ejemplo

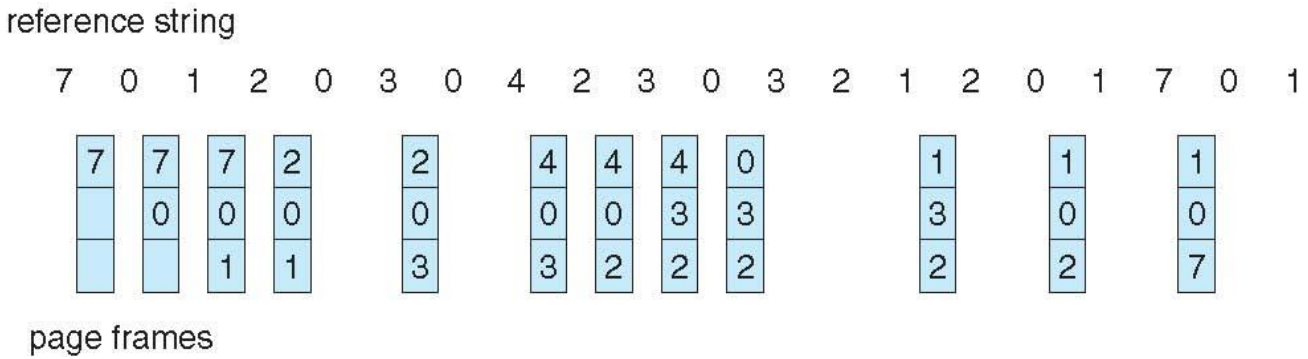
- ¿Cómo saber esto?
- No se puede leer el futuro
- Se utiliza para medir y comparar qué tan bien funcionan los otros algoritmos.





Algoritmo menos utilizado recientemente (LRU)

- Utiliza el conocimiento pasado en lugar del futuro.
- Reemplaza la página que no se ha utilizado durante la mayor cantidad de tiempo
- Asocia hora del último uso de cada página



- 12 fallos: mejor que FIFO pero peor que OPT
- Algoritmo generalmente bueno y de uso frecuente.
- ¿Pero cómo implementarlo?



Algoritmo LRU (cont.)

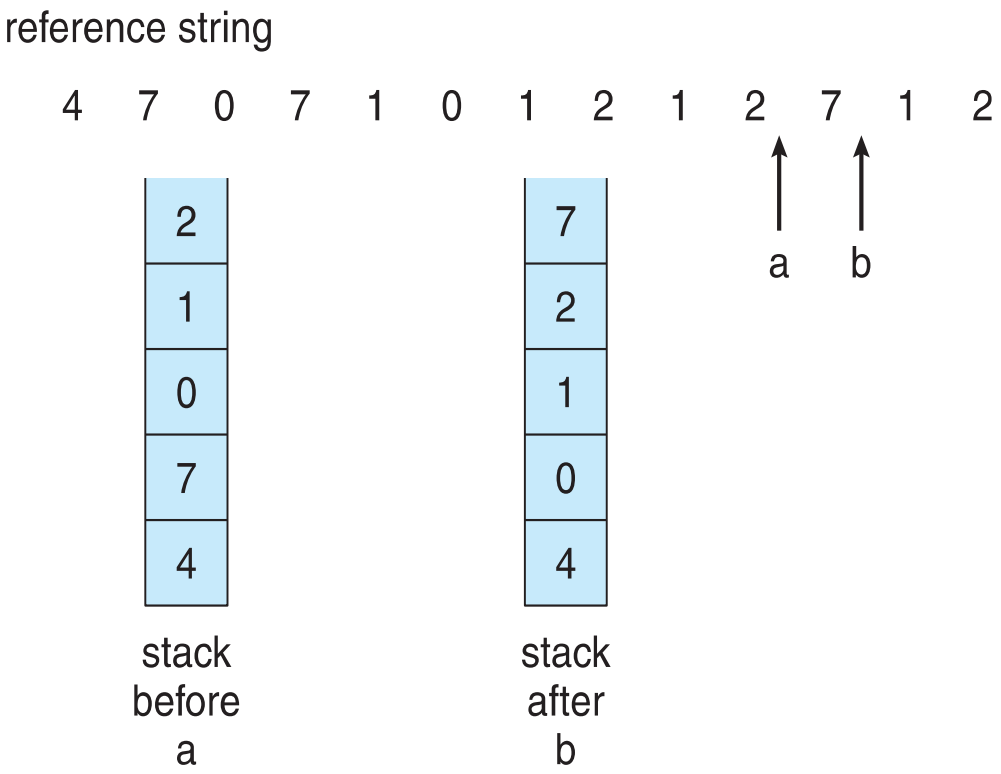
- Implementation del Contador
  - Cada entrada de página tiene un contador; cada vez que se hace referencia a una página a través de esta entrada, registra el tiempo en que ocurrió en el contador.
  - Cuando es necesario cambiar una página, mira los contadores para encontrar el valor más pequeño
    - Busca en la tabla necesaria
  - Implementación a través del Stack
    - Mantiene una pila de números de página en forma de doble enlace:
    - Página referenciada:
      - mueve al inicio del Stack
      - requiere 6 punteros para hacer los cambios
      - Pero cada actualización es más costosa
    - No busca para reemplazar





Algoritmo LRU (cont.)

- LRU y OPT son casos de algoritmos de pila que no tienen la anomalía de Belady
- Uso de una pila para registrar las referencias de páginas más recientes







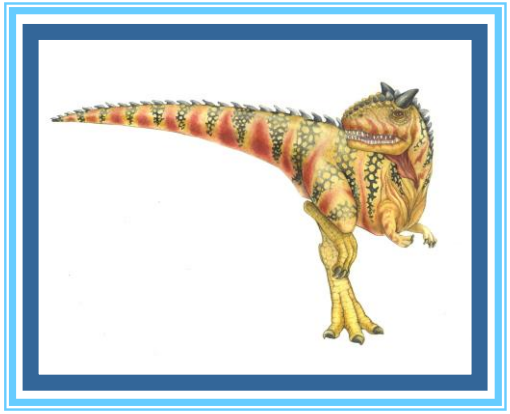
Asignación global versus local

- **Reemplazo global:** el proceso selecciona un marco de reemplazo del conjunto de todos los marcos; un proceso puede tomar un marco de otro
  - Pero el tiempo de ejecución del proceso puede variar mucho.
  - Pero un mayor rendimiento es más común.

- **Reemplazo local:** cada proceso selecciona solo entre su propio conjunto de marcos asignados
  - Rendimiento por proceso más consistente
  - Pero posiblemente una memoria infrautilizada



End of Chapter 10



Silberschatz, Galvin and Gagne ©2018